

使用 Agent Skills 做知识库检索，能比传统 RAG 效果更好吗？

mp.weixin.qq.com/s/9QT1R0vRQAV2bxLeLjifw

ConardLi

Original

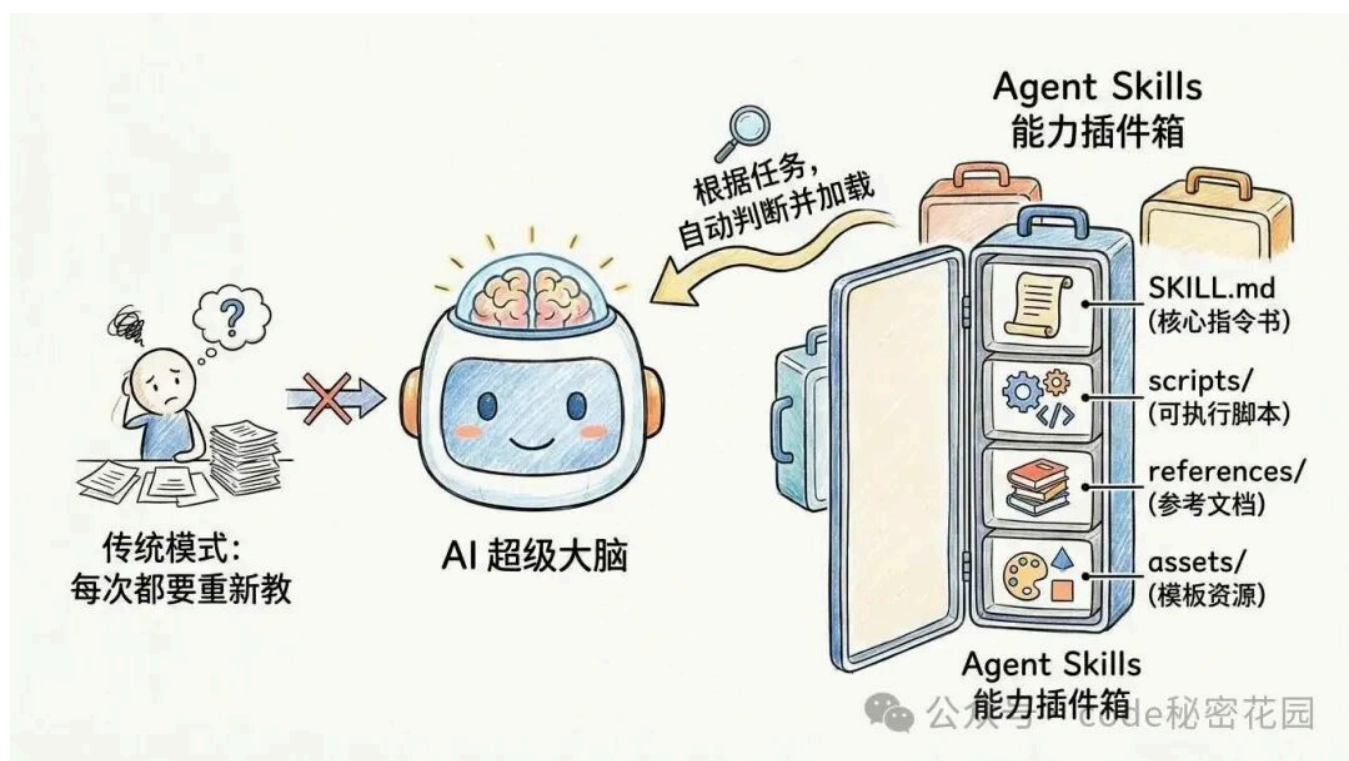
使用 Agent Skills 做知识库检索，是一种什么体验？

它能比传统的分块+向量匹配的 RAG 效果更好吗？

大家好，欢迎来到 code秘密花园，我是花园老师，这一期我们进入 Agent Skills 的实战章节。

基础回顾

我们上期视频介绍了 Skills 的工作原理和使用方法，我们简单回顾一下：

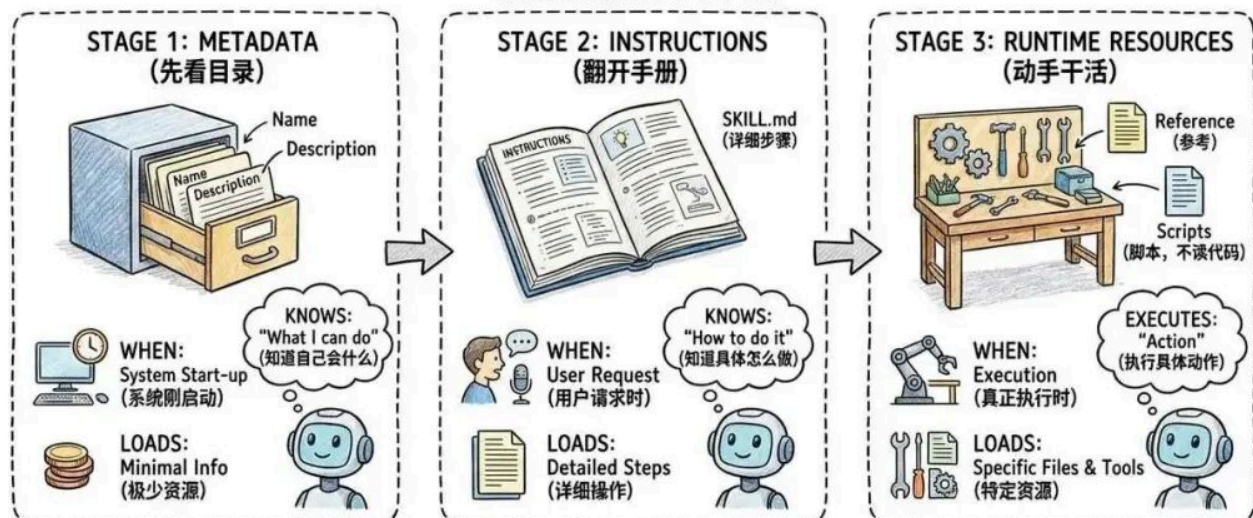


Skills 是最近 Anthropic 推出的一个 Agent 领域的行业标准，它本身就是一个文件夹，里面存放着具体的使用说明（SKILL.md），更详细的参考文档（reference），以及可执行脚本（script）。

它的核心特性就是渐进式加载策略：

SKILL: PROGRESSIVE DISCLOSURE MECHANISM

(按需加载, 用多少拿多少)



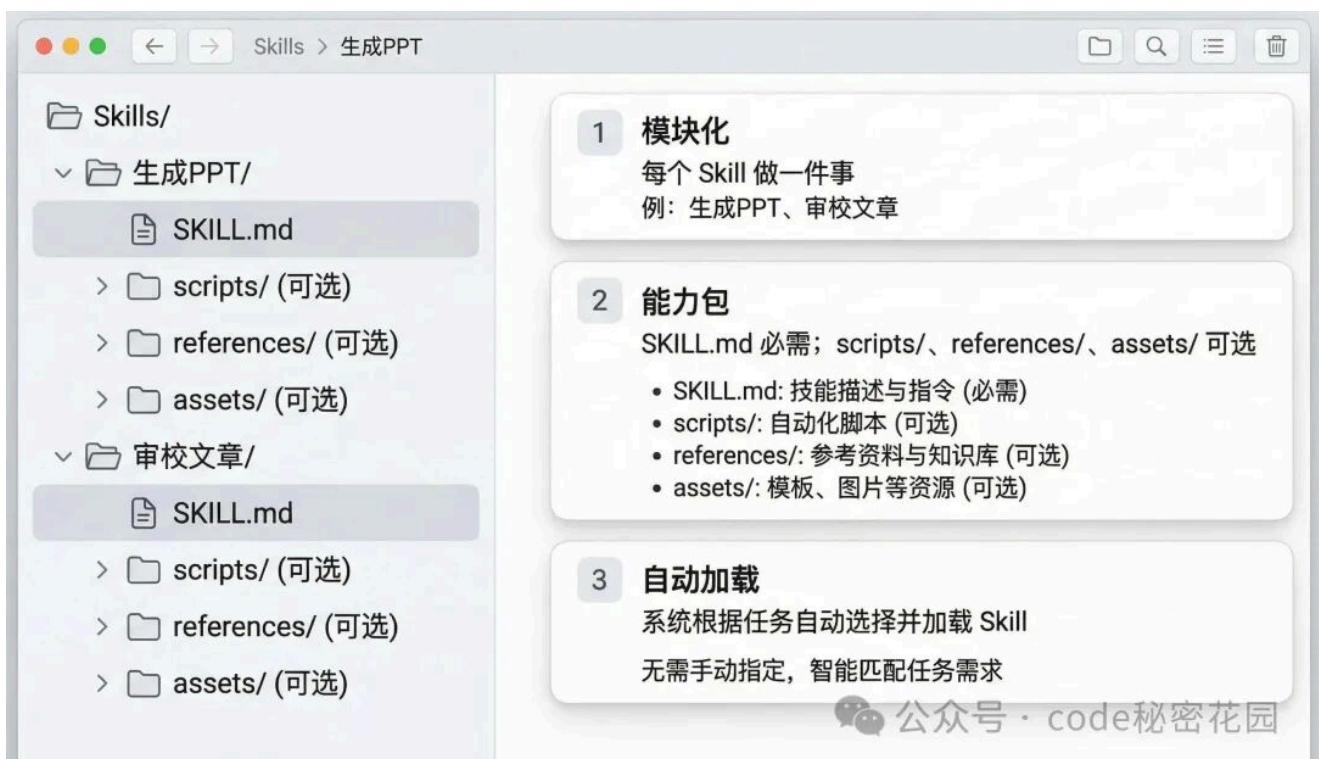
RESULT: EFFICIENT, ON-DEMAND LOADING. NO UNNECESSARY CONTEXT.

(结果: 高效, 按需加载. 无冗余上下文。)

公众号 · code秘密花园

- 在 Agent 启动时, 仅将所有 Skills 的基本描述加入 AI 的上下文
- 当用户发出需求时, AI 会根据 Skill 的描述判断具体要调用哪个 Skill
- 决定后再去读取这个 Skill 的使用说明 (SKILL.md)
- 然后再根据使用说明进一步读取更详细的参考文档, 以及决定是否执行某个脚本来连接外部世界

通过这样的模式, 可以让 Agent 既能节省大量的上下文, 又能够精准响应用户需求找到并执行某个技能。



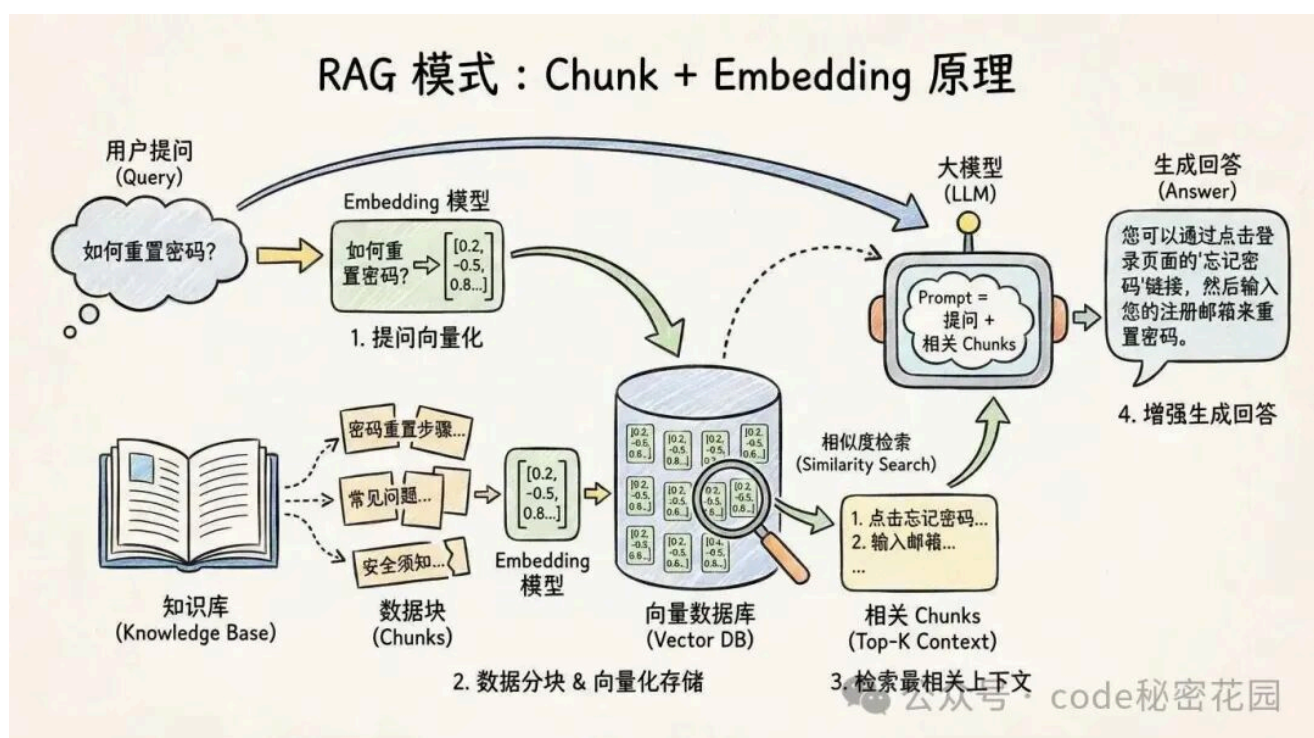
公众号 · code秘密花园

Skills 的使用也非常简单，你只需要把它的文件夹放到特定 Agent 的目录下（如 .claude/skills），AI 就会在下次启动时识别到这个 Skill，并根据用户需求判断是否调用。

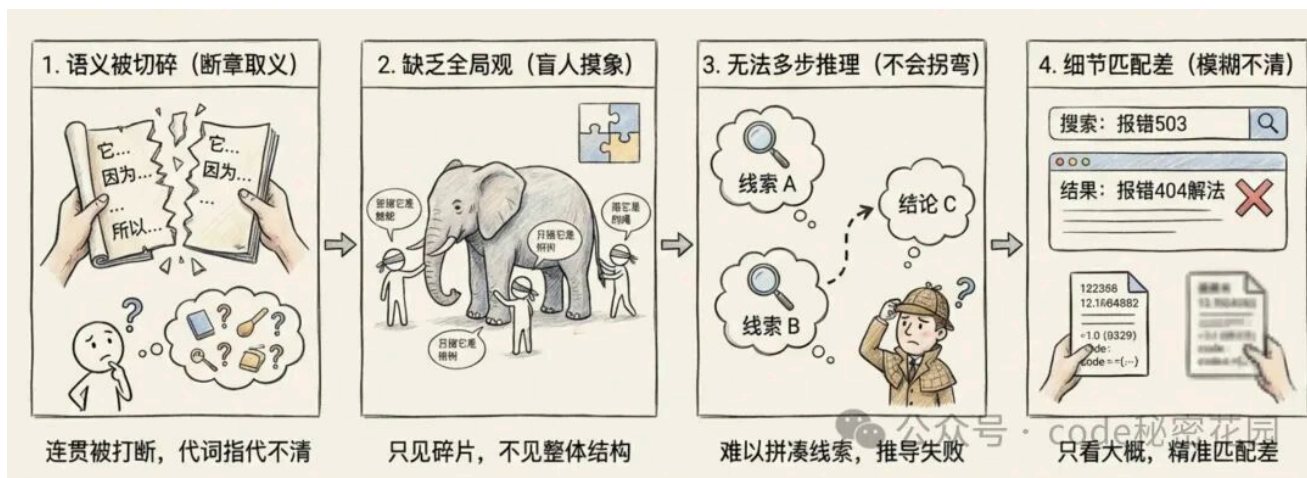
下面，我们来实战探索一下，Skill 有哪些有意思的玩法。



熟悉我的粉丝都知道，我本人对传统的通过 Chunk + Embedding 这种实现 RAG 的模式有比较大的偏见。



虽然这种模式最终也能调教出比较好的结果，但调优过程实在是太痛苦了，我是真正经历过毒打的才会有这种感觉。



相信很多同学也感同身受，所以我一直感觉这种方案只是当下的一种妥协模式，终究会被时代淘汰的。

最近 LlamaIndex 创始人 Jerry Liu 发表的 Twitter 刚好比较符合我的观点：

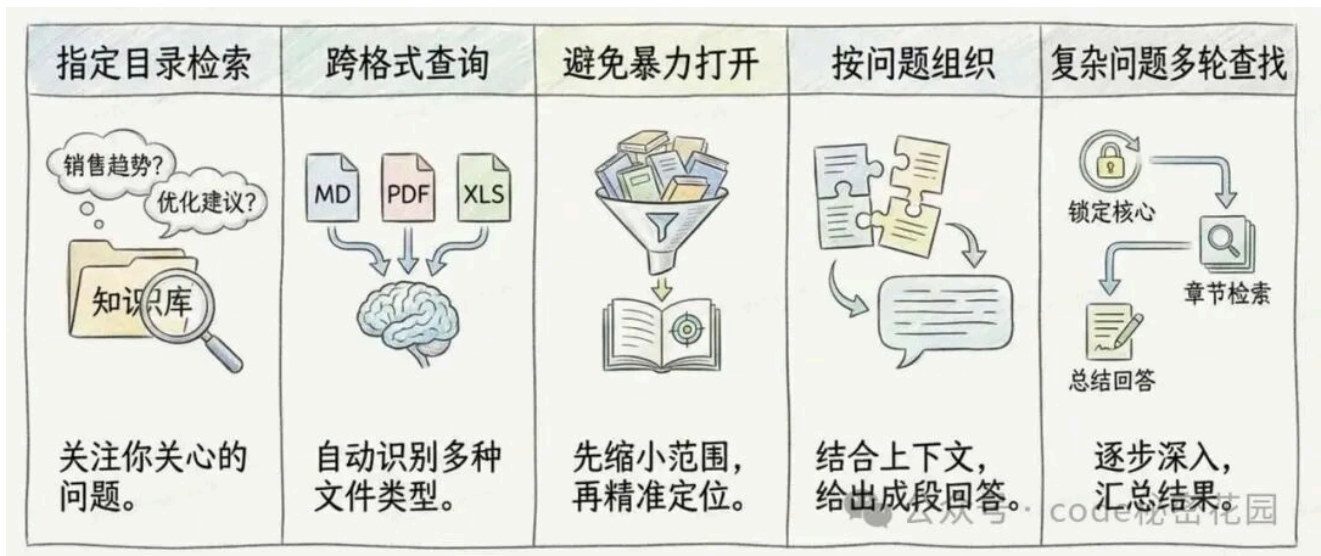


RAG / 检索本身没死，但死的是固定 Chunk + Embedding 那套模式。如果 Agent 可以动态地扩展文件周围的上下文，那么过度考虑数据块大小就没有意义了。

预期目标

想象 Agent Skills 的渐进式批漏策略，是不是有点这个意思呢。

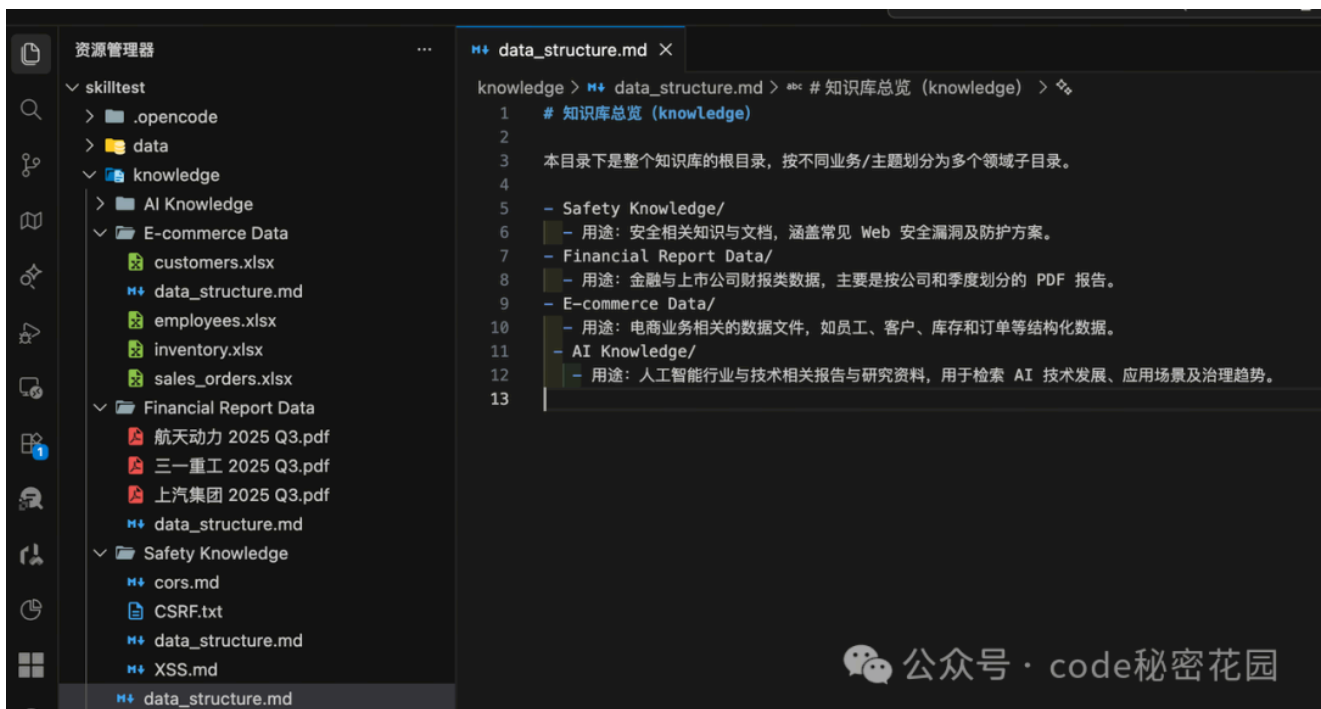
所以，我们考虑用 Skills 的设计模式，来实现一个专用于知识检索的 Skill，我期望它能帮我解决下面的问题：



- **在指定目录里检索你关心的问题**：比如：「从公司内部知识库找出最近三年的销售趋势结论」「从某个项目资料里找出所有性能优化建议」。
- **跨多种文件格式联合查询**：能根据文件类型选择合适的方式来读取和理解，比如 Markdown、PDF、Excel 等，不需要你操心「这份资料是什么格式」。
- **避免「暴力打开整库」**：不能一上来就把所有文件全部读进来，而是像人一样：先按关键词缩小范围，再精准打开少量相关文件，快速定位答案。
- **按问题来组织结果**：不能简单地给你一堆「命中行」，而是结合上下文，把和问题相关的内容拼在一起，尽量给出一条可读的、成段的回答。
- **在复杂问题上做多轮查找**：比如你问的是「对某个政策的关键影响」，它可能先在目录里锁定几份核心 PDF，再在这些 PDF 里按章节检索，再结合结果总结回答。

效果演示

下面，我们先来演示下效果，我们先创建一个示例知识库，其中包含下面这些数据：

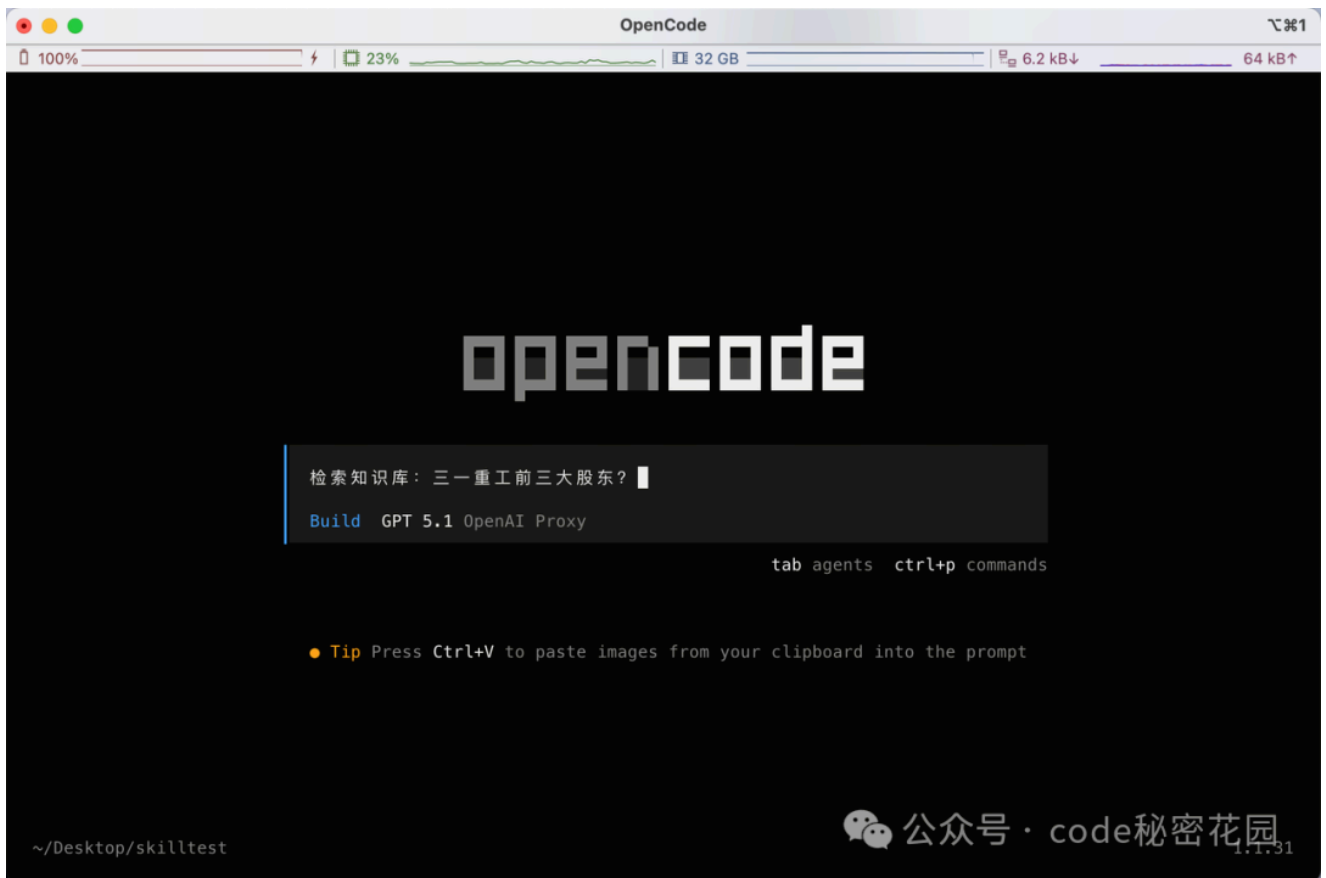


知识库包含四个不同领域的知识，包含 markdown、pdf、excel 等多种文件类型：

- Financial Report Data：金融与上市公司财报类数据，主要是按公司和季度划分的 PDF 报告。
- E-commerce Data：电商业务相关的数据文件，如员工、客户、库存和订单等结构化数据。
- Safety Knowledge：安全相关知识与文档，涵盖常见 Web 安全漏洞及防护方案。
- AI Knowledge：AI 行业与技术相关报告与研究资料，用于检索 AI 技术发展、应用场景及治理趋势。

我们先问一个金融领域具体的数据问题：

检索知识库：三一重工前三大股东？



这个数据存储在三一重工的 Q3 财报内：

二、股东信息

(一) 普通股股东总数和表决权恢复的优先股股东数量及前十名股东持股情况表

单位：股

报告期末普通股股东总数	452,943	报告期末表决权恢复的优先股股东总数（如有）	0			
前 10 名股东持股情况（不含通过转融通出借股份）						
股东名称	股东性质	持股数量	持股比例(%)	持有有限售条件股份数量	质押、标记或冻结情况	
					股份状态	数量
三一集团有限公司	境内非国有法人	2,480,088,257	29.27	0	质押	422,627,942
香港中央结算有限公司	其他	1,091,028,460	12.87	0	未知	
梁稳根	境内自然人	235,840,517	2.78	0	无	
中国证券金融股份有限公司	国有法人	233,349,259	2.75	0	无	
中国工商银行—上证 50 交易型开放式指数证券投资基金	其他	113,293,150	1.34	0	无	

检索结果如下，因为首次需要定位文件、转换 PDF 文件，耗时还是比较长的，但是检索到的信息非常准确，大家在实际使用的时候可以把所有的 PDF 都提前转成 Markdown，效率会高很多。

```
# 知识库检索：三一重工前三大股东
20,859 ($0.00) v1.1.31

* Grep "前.*三.*大.*股东|前.*三.*大.*股东|股东情况|股本及股东" in knowledge/Financial Report Data
→ Read knowledge/Financial Report Data/三一重工 2025 Q3.txt [limit=400]
* Grep "股东|股本|股份|前10名股东|前十名股东|股东情况" in knowledge/Financial Report Data (73 matches)
→ Read knowledge/Financial Report Data/三一重工 2025 Q3.txt [offset=420, limit=260]

根据知识库中《三一重工股份有限公司 2025 年第三季度报告》（前 10 名股东持股情况表）：

截至 2025 年三季度末，三一重工前三大股东为：

1. 三一集团有限公司
  - 持股数量：2,480,088,257 股
  - 持股比例：约 29.27%

2. 香港中央结算有限公司
  - 持股数量：1,091,028,460 股
  - 持股比例：约 12.87%

3. 梁稳根
  - 持股数量：235,840,517 股
  - 持股比例：约 2.78%

以上数据来源：knowledge/Financial Report Data/三一重工 2025 Q3.pdf 转出的文本中“二、股东信息 / 前 10 名股东持股情况表”。

Build · gpt-5.1-2025-11-13 · 48.1s

Build GPT 5.1 OpenAI Proxy
```

接下来我们继续追问这个文件的问题（介绍三一重工的母公司资产负债表），速度就会快很多了：



```
# 知识库检索：三一重工前三大股东 23,033 ($0.00) v1.1.31

介绍三一重工的母公司资产负债表

* Grep "母公司资产负债表|母公司 资产负债表|母公司资产负" in knowledge/Financial Report Data (1 matches)
- Read knowledge/Financial Report Data/三一重工 2025 Q3.txt [offset=1830, limit=260]

从知识库中《三一重工股份有限公司 2025
年第三季度报告》的“母公司资产负债表”部分，可以概括出母公司层面的资产负债结构大致如下（单位：千元，币种：人民币，日期：2025-09-
30，未经审计）：

一、整体规模
- 资产总计：约 43,925,878
- 其中流动资产合计：约 17,856,347
- 非流动资产合计：约 26,069,531

二、流动资产构成（母公司）
主要项目及期末数（括号内为 2024-12-31 对比数，均为千元）：
- 货币资金：5,623,570（对比：10,862,699）
- 交易性金融资产：2,786,633（302,687）
- 应收票据：24,384（28,521）
- 应收账款：2,091,216（1,888,061）
- 应收款项融资：29,366（50,567）
- 预付款项：50,971（49,844）
- 其他应收款：1,112,593（2,575,021），其中包含应收利息、应收股利等
- 存货、合同资产、持有待售资产、一年内到期的非流动资产、其他流动资产等合计后，使流动资产合计约为 17,856,347（对比：21,961,834）

三、非流动资产构成
- 债权投资及其他债权投资：其中其他债权投资约 46,939

Build GPT 5.1 OpenAI Proxy
```

接下来，我们问另外一个文件的问题：

航天动力前十股东总持股？


```
# 知识库检索：三一重工前三大股东 29,404 ($0.00) v1.1.31

航空动力前十股东总持股

* Glob "**/航空动力 2025 Q3.*" in knowledge/Financial Report Data (1 matches)
* Grep "航空动力股份有限公司 2025 年第三季度报告" in knowledge/Financial Report Data

# Convert Aerospace Power 2025 Q3 PDF to text

$ pdftotext "knowledge/Financial Report Data/航空动力 2025 Q3.pdf" "knowledge/Financial Report Data/航空动力 2025 Q3.txt" 2>&1 ||
echo "PDFTOTEXT_ERROR"

* Grep "航空动力股份有限公司 2025 年第三季度报告|航空动力股份有限公司" in knowledge/Financial Report Data
* Glob "**/航空动力*.txt" in knowledge/Financial Report Data (1 matches)
* Grep "股东信息|前 10 名股东持股情况|前十名股东|前 10 名股东" in knowledge/Financial Report Data (13 matches)
→ Read knowledge/Financial Report Data/航空动力 2025 Q3.txt [offset=380, limit=260]
→ Read knowledge/Financial Report Data/航空动力 2025 Q3.txt [offset=520, limit=260]

根据知识库中《陕西航空动力高科技股份有限公司 2025 年第三季度报告》“前 10 名股东持股情况表”：

前十名股东合计持股为：

- 合计持股数量：307,450,960 股

Build GPT 5.1 OpenAI Proxy 公众号 · code秘密花园 tab agents ctrl+p commands
```

这时我们观察一下它的执行流程（整个过程是非常智能化的）：

- 首先通过 Glob 匹配“航空动力”这几个关键字，定位到文件
- 然后调用 Python 工具将航空动力财报的 PDF 文件处理成 txt
- 接下来根据用户的问题，自己拆解出需要匹配的关键字（股东信息、前 10 名股东 ...）
- 根据初步匹配到的信息，去文件特定行数检索（offset=380 limit 260）
- 然后发现这些信息不足以回答问题，再检索更多上下文（offset=520 limit=260）

下面，我们尝试问另外一个电商领域的问题：

郑雪买了啥？

要回答这个问题需要分析两个 Excel 表：

#	A	B	C	D	E	F	G	H	#	A	B	C	D	E	F	G	H	I	J
27	C-0026	孙敏	企业	南京	江苏	2024/5/10	user0026@example.com		29	SO-10028	2025/1/28	C-0028	办公椅	办公	5	699	3495	西北	已完成
28	C-0027	黄宇	个人	苏州	江苏	2024/5/15	user0027@example.com		30	SO-10029	2025/1/29	C-0029	儿童绘本	图书	6	42	252	华东	退款中
29	C-0028	高楠	企业	成都	四川	2024/5/20	user0028@example.com		31	SO-10030	2025/1/30	C-0030	移动硬盘	电子	1	349	349	华北	已取消
30	C-0029	何洁	个人	武汉	湖北	2024/5/25	user0029@example.com		32	SO-10031	2025/1/31	C-0031	无线鼠标	电子	2	89	178	华南	已完成
31	C-0030	邱强	企业	重庆	重庆	2024/5/30	user0030@example.com		33	SO-10032	2025/2/1	C-0032	蓝牙耳机	电子	3	229	687	华中	已完成
32	C-0031	杨帆	个人	天津	天津	2024/6/4	user0031@example.com		34	SO-10033	2025/2/2	C-0033	机械键盘	电子	4	479	1916	西南	已完成
33	C-0032	郑雪	企业	西安	陕西	2024/6/9	user0032@example.com		35	SO-10034	2025/2/3	C-0034	保温杯	日用	5	39.9	199.5	东北	退款中
34	C-0033	罗旭	个人	青岛	山东	2024/6/14	user0033@example.com		36	SO-10035	2025/2/4	C-0035	咖啡豆	食品	6	78	468	西北	已取消
35	C-0034	蒋雯	企业	长沙	湖南	2024/6/19	user0034@example.com		37	SO-10036	2025/2/5	C-0036	瑜伽垫	运动	1	129	129	华东	已完成
36	C-0035	韩涛	个人	合肥	安徽	2024/6/24	user0035@example.com		38	SO-10037	2025/2/6	C-0037	护肤乳	美妆	2	159	318	华北	已完成
37	C-0036	彭蕾	企业	沈阳	辽宁	2024/6/29	user0036@example.com		39	SO-10038	2025/2/7	C-0038	办公椅	办公	3	699	2097	华南	已完成
38	C-0037	谢楠	个人	哈尔滨	黑龙江	2024/7/4	user0037@example.com		40	SO-10039	2025/2/8	C-0039	儿童绘本	图书	4	42	168	华中	退款中
39	C-0038	朱晓	企业	厦门	福建	2024/7/9	user0038@example.com		41	SO-10040	2025/2/9	C-0040	移动硬盘	电子	5	349	1745	西南	已取消
40	C-0039	林彤	个人	宁波	浙江	2024/7/14	user0039@example.com		42	SO-10041	2025/2/10	C-0041	无线鼠标	电子	6	89	534	东北	已完成
41	C-0040	梁浩	企业	石家庄	河北	2024/7/19	user0040@example.com		43	SO-10042	2025/2/11	C-0042	蓝牙耳机	电子	1	229	229	西北	已完成
42	C-0041	王晨	个人	上海	上海	2024/7/24	user0041@example.com		44	SO-10043	2025/2/12	C-0043	机械键盘	电子	2	479	958	华东	已完成
43	C-0042	李颖	企业	北京	北京	2024/7/29	user0042@example.com		45	SO-10044	2025/2/13	C-0044	保温杯	日用	3	39.9	119.7	华北	退款中
44	C-0043	张磊	个人	深圳	广东	2024/8/3	user0043@example.com		46	SO-10045	2025/2/14	C-0045	咖啡豆	食品	4	78	312	华南	已取消
45	C-0044	赵婷	企业	广州	广东	2024/8/8	user0044@example.com		47	SO-10046	2025/2/15	C-0046	瑜伽垫	运动	5	129	645	华中	已完成
46	C-0045	周杰	个人	杭州	浙江	2024/8/13	user0045@example.com		48	SO-10047	2025/2/16	C-0047	护肤乳	美妆	6	159	954	西南	已完成
47	C-0046	孙敏	企业	南京	江苏	2024/8/18	user0046@example.com		49	SO-10048	2025/2/17	C-0048	办公椅	办公	1	699	699	东北	已完成
48	C-0047	黄宇	个人	苏州	江苏	2024/8/23	user0047@example.com		50	SO-10049	2025/2/18	C-0049	儿童绘本	图书	2	42	84	西北	退款中
									51	SO-10050	2025/2/19	C-0050	移动硬盘	电子	3	349	1047	华东	已取消
									52	SO-10051	2025/2/20	C-0051	无线鼠标	电子	4	89	356	华北	已完成
									53	SO-10052	2025/2/21	C-0052	蓝牙耳机	电子	5	229	1145	华南	已完成
									54	SO-10053	2025/2/22	C-0053	机械键盘	电子	6	479	2874	华中	已完成
									55	SO-10054	2025/2/23	C-0054	保温杯	日用	1	39.9	39.9	西南	退款中
									56	SO-10055	2025/2/24	C-0055	咖啡豆	食品	2	78	156	东北	已取消

- 通过顾客表找到名叫郑雪的客户 ID
- 在订单表根据客户 ID 找到购买的商品

看一下分析结果，依然准确找到了数据：

OC | 知识库检索：郑雪买了啥？

知识库检索：郑雪买了啥？18,965 (\$0.00) v1.1.31

PY

TARGET_IDS C-0012,C-0032,C-0052,C-0072,C-0092,C-0112,C-0132,C-0152,C-0172,C-0192

HEADER order_id order_date customer_id product category quantity unit_price total_amount region status

ROW SO-10012 2025-01-12 00:00:00 C-0012 蓝牙耳机 电子 1 229 229 西南 已完成

ROW SO-10032 2025-02-01 00:00:00 C-0032 蓝牙耳机 电子 3 229 687 华中 已完成

ROW SO-10052 2025-02-21 00:00:00 C-0052 蓝牙耳机 电子 5 229 1145 华南 已完成

ROW SO-10072 2025-03-13 00:00:00 C-0072 蓝牙耳机 电子 1 229 229 华北 已完成

ROW SO-10092 2025-04-02 00:00:00 C-0092 蓝牙耳机 电子 3 229 687 华东 已完成

ROW SO-10112 2025-04-22 00:00:00 C-0112 蓝牙耳机 电子 5 229 1145 西北 已完成

ROW SO-10132 2025-05-12 00:00:00 C-0132 蓝牙耳机 电子 1 229 229 东北 已完成

ROW SO-10152 2025-06-01 00:00:00 C-0152 蓝牙耳机 电子 3 229 687 西南 已完成

...

Click to expand

根据知识库里的电商数据，名为“郑雪”的客户（有多位同名，不同 customer_id）在销售订单中购买的商品是：

- 商品：蓝牙耳机

- 类目：电子

- 订单数量：每笔订单 1-5 副不等

- 单价：229 元

- 订单时间：从 2025-01-12 到 2025-07-11 期间多次下单

也就是说，在当前销售订单数据里，“郑雪”只买过蓝牙耳机，没有出现其他商品记录。

Build · gpt-5.1-2025-11-13 · 39.5s

Build GPT 5.1 OpenAI Proxy

公众号 · code秘密花园

tab agents ctrl+p commands

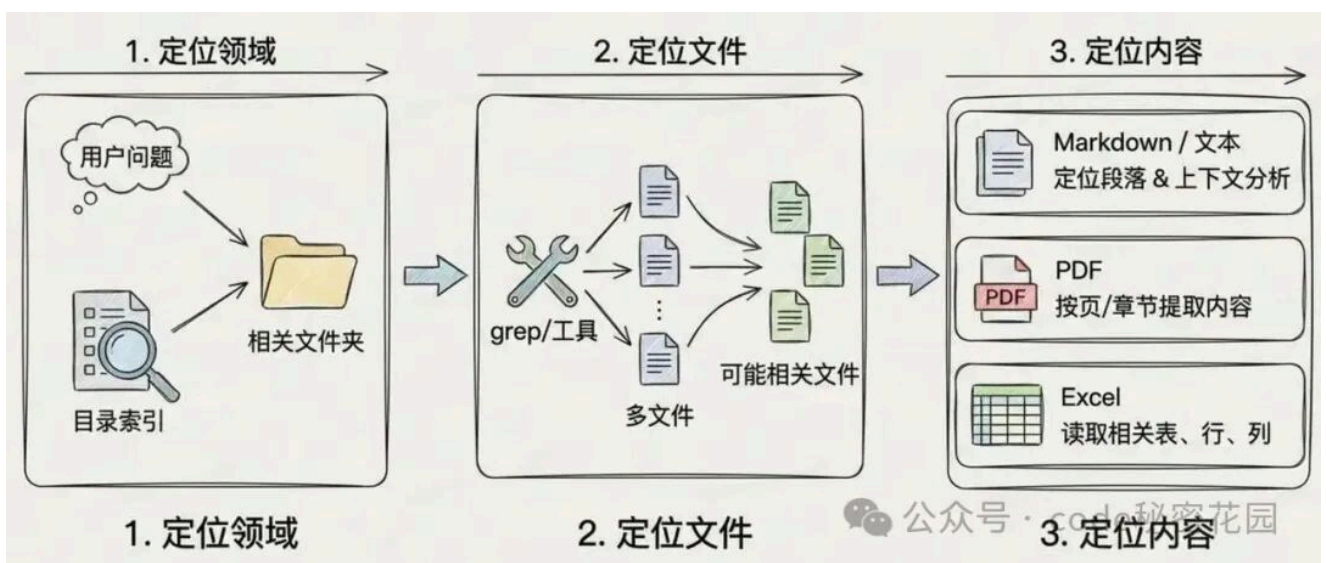
我们再追问一个更复杂的问题：有哪些用户买了 儿童绘本，在什么时间？



这次 Agent 更快的给出了结果，这个过程已经比较接近于我理想的 AI 智能知识检索的形态了，大家觉得怎么样呢。

原理介绍

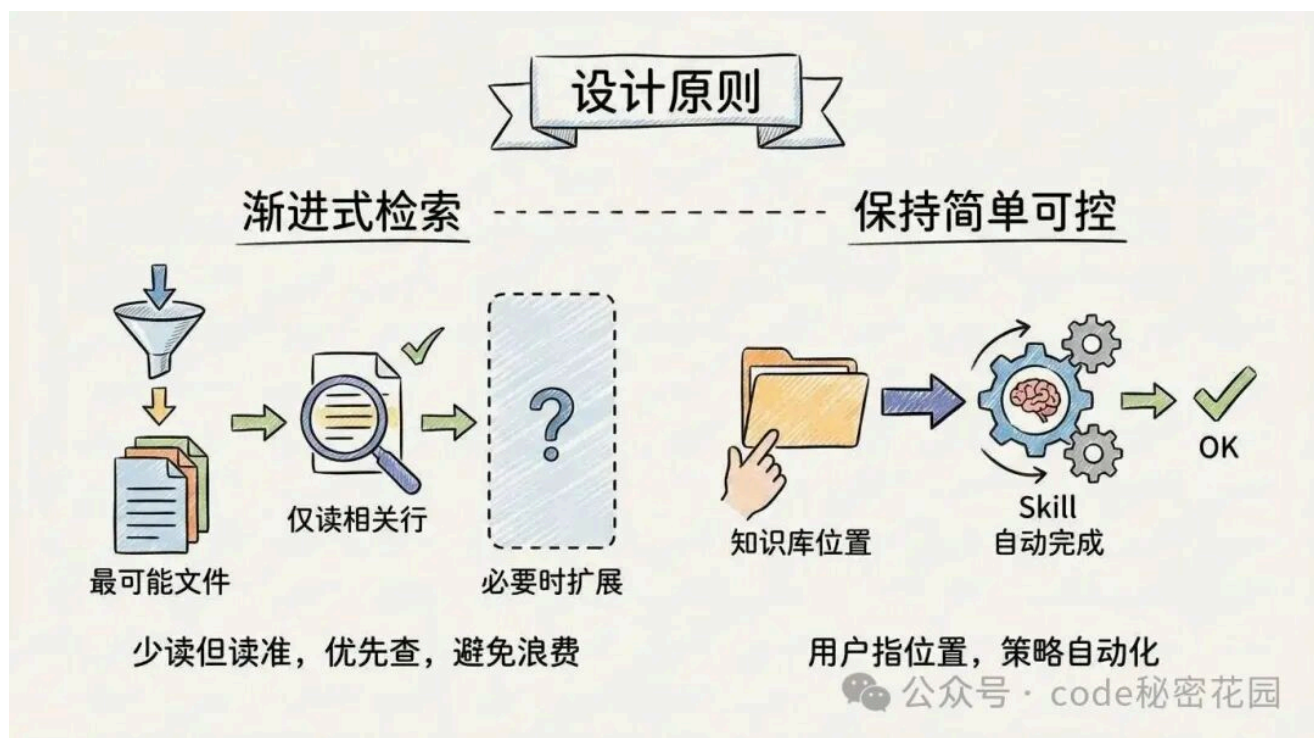
下面，我们来看看这个 Skill 的实现原理，对于一个能执行 Skill 的 Agent (如 opencode)，本身它已经具备了一些基础的文件检索能力 (ls、grep、glob)，我们只需要在这个 Skill 中教会 AI 如何更好的使用这些能力：



- **定位领域**：通过分析用户的问题，然后结合知识库的目录索引定位出知识可能存在的领域（文件夹）。

- **定位文件**：使用 grep 这样的方式在指定目录、文件里筛选出「可能相关的文件」，然后再按需用不同工具去读取这些文件的关键部分。
- **定位内容**：针对不同文件类型用不同策略：
 - **Markdown / 文本**：直接定位到匹配段落，结合上下文分析
 - **PDF**：编写代码调用专门的 PDF 能力按页、按章节提取内容
 - **Excel**：编写代码读取 Excel，然后只看跟问题相关的表、行、列，而不是把整份表格搬出来

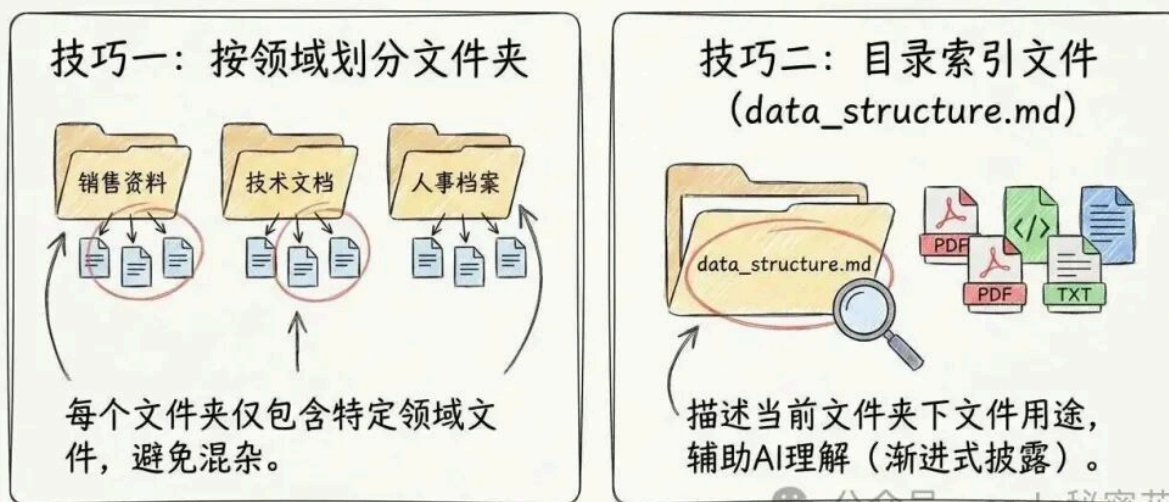
这里有条关键设计原则：



- **渐进式检索**：尽量少读但读准，优先查「最可能有答案」的文件，读取文件内容时仅读取相关行，必要时再扩展范围，避免无意义的 Token 消耗。
- **保持简单可控**：用户只需要告诉它知识库在什么位置，其余的检索策略都由 skill 自动完成。

另外想要让知识检索的更高效和精准，还有两个技巧：

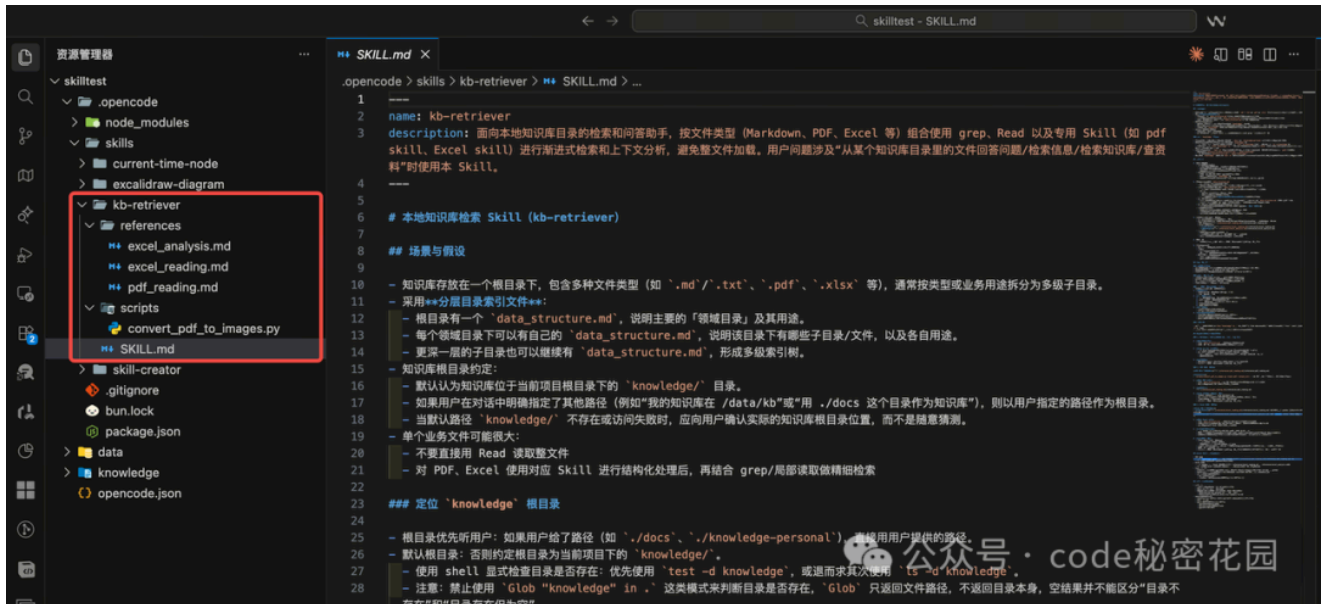
高效精准的知识检索技巧（手绘指南）



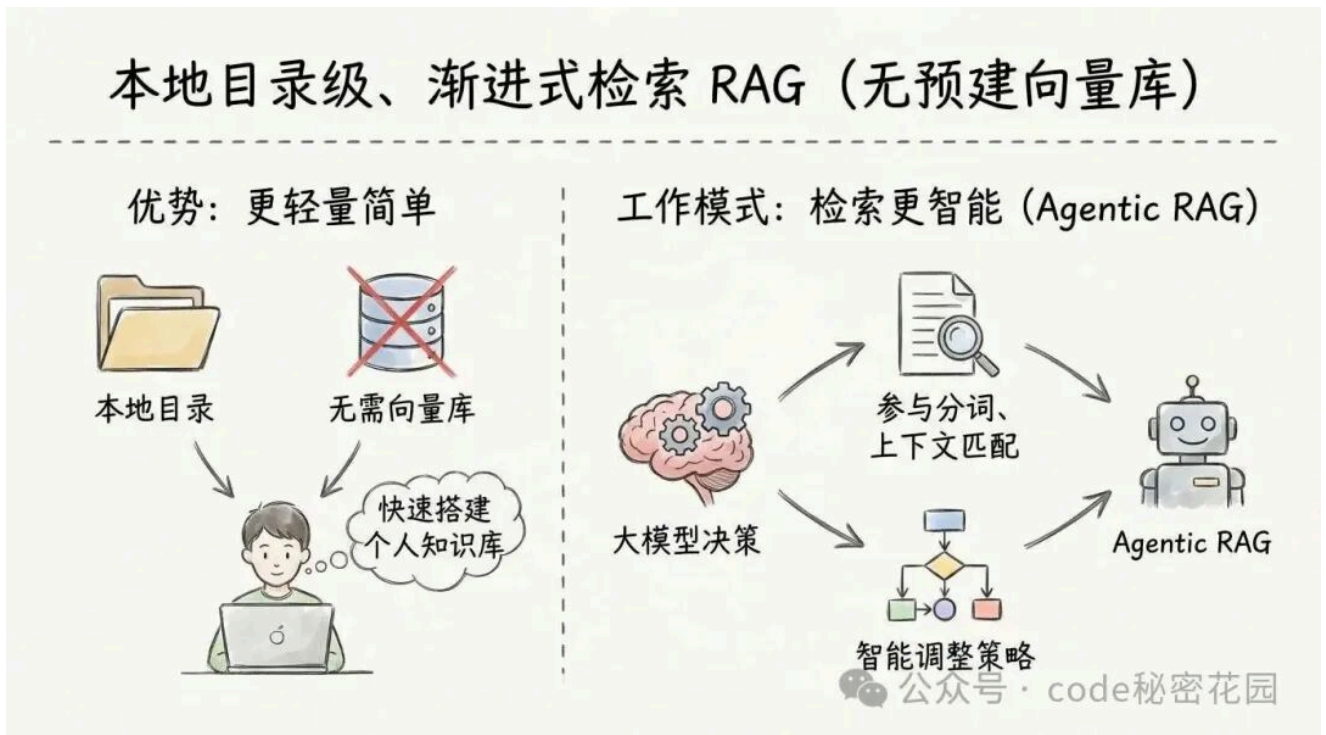
- 文件夹尽量按领域划分，每个文件夹下只包含特定领域下的文件
- 每个文件夹下都有一个目录索引文件 `data_structure.md`（如果文件命名不规范、且包含 PDF 等非纯文本的文件格式，这个是必要的），这个文件中描述了当前文件夹下每个文件的用途（这里参考了 Skill 的渐进式披露的设计原则，如果有多级嵌套领域（目录），AI 找到了这个领域后才会读取这个领域的目录索引）。

那这个 Skill 怎么实现呢，其实你把上面我们提到的核心流程、设计原则、关键技巧给到我们上个教程中讲到的 skill-creator 这个 skill，AI 自己就帮我们创建出来了，我们只需要经过细微的调整，下面我们来看下我们的 Skill 的目录结构：

- **SKILL.md**：是这个技能的使用说明书，里面描述了如何找到本地知识库目录、如何根据目录索引逐层检索知识、对不同文件的处理方式（参考哪个文档）、最终的回答风格等等
- **references**：放置 PDF、Excel 等特殊文件类型的解析方法，只有确定当前知识需要具体分析一个 PDF、Excel 或其他特殊的文件类型时，再查阅详细的处理文档，可以节省 Token 和注意力。
- **scripts**：放置 PDF 文件的特殊处理脚本（如解析 PDF、PDF 转图片等等）



我们可以把这套方案可以理解为「不预建向量库的、本地目录级、渐进式检索 RAG」，的优势主要在于：

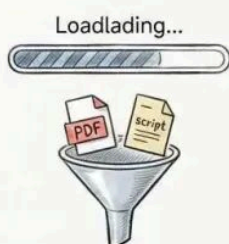


- 不需要预建索引 / 向量数据库，更轻量简单，适合快速搭建本地个人知识库的场景；
- 检索更智能，传统的 RAG 主要靠向量匹配内容，大模型只做总结，而在这个过程里，大模型可以参与分词、上下文匹配的决策，并且可以进行智能调整，其实就是一个 Agentic RAG 的工作模式。

当前这只是一个初步的尝试，它还有一定的缺陷：

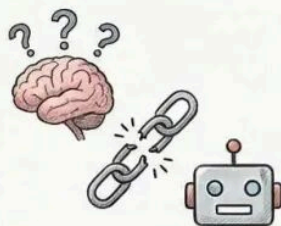
缺陷与挑战

首次检索效率低



特殊格式（如PDF）需转换，耗时。推荐预处理为纯文本。

Skill调用不稳定



多轮检索后AI可能忘记调用Skill，丢失关键步骤。

Token消耗更大



AI控制检索过程，未找到答案会反复尝试，更耗Token。

公众号：code秘密花园

- 首次检索的效率问题：如果分析到的文件是一个 PDF 或其他特殊格式，需要先调用脚本进行转换，首次效率低（推荐预处理为纯文本）
- Skill 调用稳定性：在多轮检索后，AI 可能会忘记去调用 Skill（第一次命中率还是比较高的），从而丢失一些关键的处理步骤，这个在很多其他的 Skill 上也会有类似的问题，可能需要从 Agent 层面解决。
- Token 消耗更大：因为整个检索过程是 AI 控制的，当首次没有检索到想要的答案，AI 会继续切换一些新的参数或文件进行检索，直到找到最终结果，虽然结果更准确，但也更消耗 Token。

让已有文档站秒变知识库

刚刚我们介绍的 Skill 是一个通用的知识检索的尝试，但如果你有一个固定格式的知识库（比如一个文档站，里面已经包含了多个本地文件），你可以通过 Skill Creator 让它去自己分析你的文档站，然后创建出符合你的文档站风格的知识检索 Skill，我们可以在一个已有文档站下输入下面的提示词：

当前项目是一个文档站（包含多个 md、mdx 文件），我想创建一个专用于知识库检索的 Skill，它可以替代传统的 RAG，用于当前文章站的知识检索，你来帮我创建这个 Skill（使用中文）。Skills 的基本思路是，先根据用户的问题，提取出需要检索的可能关键字，然后使用 Grep 命令找到对应的上下文。然后分析上下文，如果检索到的信息和问题不符合需要继续检索其他可能关键字，最大重试五次。


```
# 中文知识库检索 Skill 设计与思路要点
9,355 ($0.00) v1.1.35

帮我创建一个专用于本次知识库检索的 Skill，当前项目是一个文档站（包含多个 md、mdx 文件），这个 Skill 可以替代传统的 RAG，用于当前文档站的知识检索，你来帮我创
造这个 Skill（使用中文）。Skills 的基本思路是，先根据用户的问题，提取出需要检索的可能关键字，然后使用 G
rep 命令找到对应的上下文，然后分析上下文，如果检索到的信息和问题不符合需要继续检索其他可能关键字，最大重试五次。

@ skill [name=skill-creator]

下面是一个可以直接作为 Skill 使用的 SKILL.md 草稿（中文），实现你说的：基于 Grep 的多轮关键词检索，最多 5 轮，用于当前文档站（md/mdx）的知识检索，替代传统向量 RAG。
你可以把下面内容保存为某个 Skill 目录里的 SKILL.md，比如 .opencode/skills/kb-grep-docs/SKILL.md，然后按你现有流程打包/启用。

---
name: kb-grep-docs
description: 基于当前 Git 仓库中的 Markdown/MDX 文档，使用关键词 + Grep 进行多轮上下文检索和问答的知识库 Skill，用于替代传统向量
RAG。适用于回答“这个文档站里有没有...”“某个配置/API 怎么用”“某章节怎么说明...”等与本仓库文档内容直接相关的问题。
---

基于 Grep 的文档检索 Skill

本 Skill 用于在当前项目的文档站中（大量 `.md` / `.mdx` 文件）执行**轻量级、高精度的多轮检索**，替代向量 RAG。核心思路：

- 从用户问题中提取多个可能的检索关键词/短语
- 使用 Grep 在 Markdown/MDX 文档中搜索匹配行
- 根据匹配结果构造上下文，判断是否足以回答问题
- 如不够，调整关键词继续检索，最多重试 5 轮
- 最终用检索到的真实文档内容回答；如果确实找不到，要明确说明

---

何时触发本 Skill

在以下情况使用本 Skill：

- 问题明显在问**当前文档站的内容**，例如：
  - “这个项目支持哪些数据源？”
  - “如何在 easy-dataset-docs 里配置 xxx？”
  - “文档里对 yyy 的限制是什么？”
- 问题出现诸如“文档、手册、README、指南、配置说明、API 文档”等字样
- 问题提到仓库中可能存在的章节/文件名，如：
  - “在 `quickstart` 里怎么写的？”
  - “有没有 `dataset.mdx` 相关说明？”

以下情况**不要优先触发**本 Skill（或触发后应快速发现无结果并退出）：
```

下面尝试问个问题：

检索知识库：如何使用 Easy Dataset 生成评估数据集？

```
# Easy Dataset 评估数据集生成方法
13,124 ($0.00) v1.1.35

检索知识库：如何使用 Easy Dataset 生成评估数据集？

@ skill [name=kb-grep-retriever]
* Glob "**/*.md" (44 matches)
* Grep "ping-gu" in . (4 matches)
+ Read ping-gu/wei-tiao-xiao-guo-ping-gu.md
+ Read ping-gu/README.md

在 Easy Dataset 里，评估数据集（测试集）本质是「题目 + 标准答案/参考答案 + 评分规则/选项」的一组集合。
官方文档里给了 3 条主要生成路径，你可以按场景选用或组合使用。

一、先理解：支持哪些题型

评估数据集里的题目类型有 5 种：

- 判断题：是/否，适合测幻觉（事实是否说对了）
- 单选题：A-D 单选，考察在干扰项下的知识提取和辨析
- 多选题：答案为如 ["A","C"]，极具挑战性，漏一个就错
- 简答题（短回答）：20 字以内，测抓住核心信息、简洁表达
- 开放题（长回答）：测推理、分析、总结能力，没有唯一标准答案

在任务配置里可以设置各题型的生成比例（如 30% 判断题 + 70% 简答题）。

---

二、从领域文献自动生成评估数据集

适用于：有 PDF/Docx 报告、说明书、规范文档等，希望从中自动出题。

1. 进入 数据源 - 文献处理 模块，导入 PDF 或 Docx 文档。
2. 系统会自动解析并切分成多个文本块（Chunk）。
3. 可批量编辑这些文本块，例如在每个块前加上全局摘要信息，以便模型更好理解上下文。
4. 在文献处理界面中：
  - 可以基于单个文本块「单次生成」测试题，先检查题目质量；
  - 也可以一键「自动生成测试集」，后台会自动遍历未出题的文本块。
5. 系

■ Build · gpt-5.1-2025-11-13
```

由于这个文档站全是纯本文文档（没有 PDF 等特殊格式）检索速度非常快而起效果非常不错。

再问个问题：

检索知识库：Easy Dataset 有哪些分块策略？

检索知识库: Easy Dataset 有哪些分块策略?

```
o skill [name=kb-grep-retriever]
* Grep "分块|切分|chunk" (39 matches)
- Read basics/editor.md [limit=2000, offset=0]
```

Easy Dataset 当前支持的主要分块策略有:

- 固定长度分块
 - 按长度切分文本, 可按「字符数」或「Token 数」分块
 - 典型参数: `separator` 分隔符、`chunkSize` 最大块长、`chunkOverlap` 重叠长度
- 文本结构 (递归) 分块
 - 利用文本的自然结构 (段落 → 句子 → 单词) 递归切分
 - 支持配置多个自定义分隔符 + 最大块长 + 重叠长度
 - 适合结构较复杂、需要多级分隔符的文档
- 文档结构 (Markdown) 分块 (默认策略)
 - 基于 Markdown 章节标题 (`#` / `##` / `###` 等) 识别章节
 - 在最小 / 最大分割长度范围内按章节划分, 超长章节再递归细分
 - 适合结构良好的 Markdown 文档
- 代码结构分块 / 程序代码智能分块
 - 针对含大量代码的文档, 基于编程语言语法和代码语义智能分块
 - 避免在函数 / 类等语法单元中间粗暴截断
- 自定义分隔符分块 (含递归 / 固定长度两种形态)
 - 可以指定一组自定义分隔符 (例如特定标题、分割线 `-----`、`##` 第 等)
 - 支持「严格按分隔符断开」或「按分隔符切完后再组合成固定长度块」两种模式
- 可视化自定义分块
 - 在文献预览界面, 直接用鼠标选中需要分块的位置, 手动定义分块边界
 - 可实时查看块数量、每块字符数, 保存后会替换历史自动分块结果

所有这些策略都可以在 **设置** → **任务设置** → **分块设置** 中选择和配置。

如果你关心某一种策略的具体参数 (比如 `chunkSize/overlap` 或自定义分隔符怎么写), 我可以帮你展开详细说明或给配置示例。

■ Build · gpt-5.1-2025-11-13 · 13.7s



公众号 · code秘密花园

通过这种模式, 你可以把任意已经存在的本地知识库变成一个可以进行智能知识检索的 skill。